

PennyPilot: Permission-Gated, Hidden-Constraint Shopping with Reinforcement-Learning Post-Training on V100 Hardware

Param Madan

July 25, 2026

Abstract

We present **PennyPilot**, a multi-turn, multilingual shopping agent trained with a complete reinforcement-learning post-training pipeline (SFT $\rightarrow$ RLOO / GRPO) on a single 32 GB V100 (Volta) GPU. The task is engineered so naive strategies fail: the shopper’s budget and 2–3 hard requirements are hidden and revealed only through correct clarifying questions, and under partial discovery the cheapest visible item is a distractor in **99/100** scenarios. Rewards are computed against catalog ground truth — no learned reward model, nothing to hack — and a **structural permission gate** makes any cart addition without an explicit per-SKU grant *unrewardable*, holding safety at **0 violations** across every run. We report the end-to-end system, a **pre-registered** RLOO-vs-GRPO comparison at matched rollout budget, a 14-row systems/efficiency campaign that measured rather than assumed each optimization (**6.96** $\times$ iteration speedup), the V100-specific precision engineering that makes the loop trainable at all, and — deliberately — the negative and partial results. PennyPilot is the post-training arm of a lineage that begins with **shoprl-fabric** (a single-machine multi-turn RL platform) and points toward **Aether**, a self-hosted distributed spatial-vector search engine, as its future retrieval substrate.

1 Introduction

Public RL-post-training demonstrations tend to fail in one of two ways: (a) they train on tasks already near-saturated for the base model, so the “improvement” is indistinguishable from noise; or (b) they omit the systems engineering that decides whether the loop is trainable at all on the single academic/consumer GPU most practitioners actually have. PennyPilot addresses both. The task’s difficulty is *measured and gated* before any training spend, and the entire pipeline — rollout, reward, optimization, evaluation, checkpointing — is built and validated on one Volta-generation V100.

The agent is the *workload*; the engineering target is the *training infrastructure*: multi-turn RL stability, a safety mechanism that survives optimization pressure, and profiling-driven rollout efficiency. Our contributions:

1. A **verifiable, hidden-constraint task** whose difficulty is pre-registered and measured (not assumed), giving RL real signal.
2. A **structural (non-reward) permission gate** with 0 measured violations across SFT, RLOO, GRPO, and zero-shot transfer.
3. A **pre-registered RLOO-vs-GRPO study** (H1–H3) at matched rollout budget, including confirmed, partial, and negative outcomes.

4. A **measurement-first systems campaign** (SS0–SS13) — including optimizations deliberately *rejected* on profiling grounds.
5. The **V100 precision engineering** (fp16 + dynamic loss scaling; the reasons bf16 and LoRA-serving fail on Volta) that makes the loop viable on 32 GB.

2 System Architecture

PennyPilot is organized as three planes with a single execution path between them.

Environment plane. A frozen, deterministic **synthetic catalog** is the source of ground truth. A **scenario generator** samples hidden needs {category, hidden budget, 2–3 must-haves} and derives the exact set of satisfying SKUs (**valid_skus**) so reward is a verifiable predicate. **PennyEnv** is the conversation state machine over a structured action grammar; the **user-simulator** is split into two strictly separate seams — a swappable **ConversationModel** (phrasing only) and a rule-based **judge_accept** (the *only* accept/reject authority). The reward reads the judge’s *decision*, never the model’s *words*, so no conversation model can be persuaded into a bad accept.

Training plane. SFT warmup on environment-recorded demonstrations, then the multi-turn RL loop: reset the simulator to an identical hidden state, sample k full-conversation rollouts, score each against ground truth, compute a leave-one-out (RLOO) or group-normalized (GRPO) advantage, take a KL-controlled update against a frozen reference, checkpoint.

Infrastructure plane. Training runs under **Slurm** on a single V100; checkpoints bridge via **S3** to (designed) A10G rollout workers; **vLLM** serves the policy for rollout; per-step **metrics.jsonl** feeds an **observability dashboard** (KL, reward, throughput, GPU utilization) with KL-blowup / reward-stall alerts. Preemption safety (SIGTERM → requeue → resume) is drilled operationally.

What is deliberately *not* in the stack. There is **no learned reward model** (reward is catalog-verifiable) and **no message-queue / streaming layer** (Kafka, etc.); the loop is a single-machine synchronous pipeline by design. Distributed, streaming, fault-tolerant retrieval is the role of a *separate* system, Aether (§9), not of the post-training loop.

2.1 Structured action interface

The policy emits single-line JSON actions rather than raw browser selectors or free-form tool calls — the interface keeps the agent inspectable and the environment authoritative:

```
{
  "action": "ask_user", "question": "What is your total budget?"
}
{"action": "search", "query": "laptop under 2000 dollars"}
{"action": "select_product", "product_id": "LAP-0014", "reason": "cheapest valid"}
{"action": "request_cart_permission", "items": ["LAP-0014"], "estimated_total": 796.29}
{"action": "add_to_cart", "product_id": "LAP-0014"}
```

Environment adapters translate these into effects: the synthetic catalog (training + most eval), a **WebShop**-compatible adapter (unseen-product transfer), and a **Playwright** Chromium mirror used *only* for human-facing demo/replay — the browser is never in the training loop, only a projection of decisions already made.

3 Task Design and Data

Hidden constraints. The shopper’s objective is hidden: a budget ceiling and 2–3 hard constraints (brand, min RAM, min battery, max weight), disclosed one at a time and only in response to

a *relevant* clarifying question. Under partial discovery, the cheapest item satisfying the *known* constraints violates an *undiscovered* one in **99 of 100** generated scenarios — so “grab the first result” loses and disciplined enumeration wins. This gives the reward real texture instead of a turn-one near-ceiling pass rate.

Ground-truth rewards. Scoring is against the catalog’s real specifications, never the model’s stated beliefs — no reward model, no surface for reward hacking via plausible-but-false claims.

Data pipeline.

- *Catalog*: procedurally generated, deterministic by seed; specs mildly correlated so constraint trade-offs are non-trivial.
- *Scenarios*: hidden-need profiles with a derived `valid_skus` answer set; **valid-set size is a tuned knob** so `value_quality` (price-rank among valid) has real gradient rather than a near-binary signal.
- *SFT demonstrations are recorded from the environment itself* — env transcripts with a build-time correctness guard — not plausible-looking hand-written text.
- *Language layer*: deterministic EN / ES / code-switch detection and bilingual constraint extraction; a constraint survives whatever language it arrives in.

4 Method

4.1 Structural permission gate

`add_to_cart(SKU)` succeeds only if that exact SKU is in a previously granted permission set — enforced in the environment’s **dialogue-state transition**, not in the reward term. No weight configuration can produce a rewarded cart addition without a real matching grant first. Ambiguous replies are not approval; a held “no” survives subsequent permission requests until explicitly lifted. Because a violating trajectory never produces a legit cart, it has *no value/accepted term to offset the penalty* — the gate is structural, not a soft cost. **Measured: 0 violations across every SFT / RLOO / GRPO / transfer run.**

4.2 Reward and credit assignment

Per completed trajectory,

$$R = 0.4 \cdot \text{value_quality} + 0.4 \cdot \text{accepted} + 0.2 \cdot \text{asked_permission} - 1.0 \cdot \text{acted_without_permission} + \sum_t \text{info_gain}_t. \quad (1)$$

`value_quality` is gated to 0 on any hard-constraint violation, else graded by price-rank among valid items (cheapest = 1.0). The four *outcome* terms form a trajectory scalar that credit-assignment spreads across turns (uniform, or γ -discounted if learning stalls); **information gain** — the reduction, in bits, of the catalog set still consistent with what the agent knows — is a *per-turn* bonus on the turn a clarifying question was asked, added through the *same* credit path (no divergent second reward). Info gain is the dense signal that counters the “RL kills clarification” collapse.

4.3 Optimization: RLOO and GRPO

Both are critic-free, group-based policy-gradient methods; they differ *only* in the baseline, so a matched-budget comparison isolates the advantage rule. For a prompt with k sampled trajectories

and returns $r_1, \dots, r_k$:

$$\text{GRPO (group-normalized): } A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}, \quad (2)$$

$$\text{RLOO (leave-one-out): } A_i = r_i - \frac{1}{k-1} \sum_{j \neq i} r_j. \quad (3)$$

GRPO’s baseline is the full-group mean (which includes r_i), divided by group std; RLOO’s leave-one-out baseline is *provably unbiased* (independent of the action it scores) with *no* std rescaling. The surrogate is the same clipped policy-gradient objective with a KL penalty to a frozen reference:

$$L = -A \cdot \log \pi_\theta(\text{agent tokens}) + \beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}}). \quad (4)$$

We use the k3 (unbiased, low-variance) KL estimator and log final + max KL each step; **KL is the core stability lever**. RLOO is the bring-up algorithm; GRPO is compared at an *equal* 400-trajectory budget so any gap is attributable to the advantage rule, not sample count.

4.4 Loss-mask verification

SFT loss is on policy tokens only. A **reference-exact verifier crashes the run** if any environment- or user-authored token leaks into the loss — the failure mode where training loss improves while behavior degrades because the model is partly learning to *imitate the environment* instead of *act* in it.

5 The V100 Problem: Precision and Serving on Volta

Training this loop on a single 32 GB V100 is the load-bearing systems constraint, and Volta (compute capability 7.0) imposes three concrete obstacles.

(1) **No usable bf16**. Volta lacks bf16 tensor cores; bf16 runs *emulated* in software. Measured, **fp16 + dynamic loss scaling (GradScaler)** is **4.27×** faster than the bf16-emulated path — so fp16 is not a stylistic choice but a throughput requirement.

(2) **fp16 is unstable without scaling**. Unscaled fp16 training **diverges at step 1** on this task (gradient underflow/overflow). Dynamic loss scaling is *necessary, not optional*; with the scaler applied, fp16 is numerically stable throughout. (A full-fine-tune variant we profiled needs fp32 optimizer state, which does not fit alongside a frozen reference on 32 GB — motivating the LoRA path below.)

(3) **7B policy + reference in 32 GB**. A naive on-policy RL setup needs two model copies (policy and frozen reference). Instead, a **shared-reference trick** — one frozen base carrying two LoRA adapters (policy + reference) — lets 7B RL fit in 32 GB, halving the weight footprint.

(4) **vLLM serving is hardware-gated on Volta**. vLLM’s LoRA-serving and prefix-caching kernels require Ampere-class Triton support; on Volta they are non-functional. The workaround is **merge-at-sync serving** — merge the LoRA adapter into the base at each policy sync and serve the merged weights — which is also independently a **2.3×** rollout speedup. (In first bring-up on an A10G we also hit a bleeding-edge vLLM engine-init bug in the flashinfer sampler; the documented mitigation is version pinning / native-sampler fallback.)

6 Experimental Setup

Models. Qwen2.5-1.5B-Instruct validates the entire pipeline at low cost; Qwen2.5-7B-Instruct is the production line.

Why not stop at 1.5B? The 1.5B model is a *pipeline-validation* instrument, not the deliverable. It exists to confirm — cheaply, before any GPU-hours are committed to a larger model — that four things are correct end to end: the environment and its ground-truth reward, the SFT recipe (including the loss-mask verifier of §4.4), the RL loop (rollout → advantage → KL-controlled update), and the evaluation harness. Our *pre-registered hypothesis* was a capability split:

- **H0a (loop-provability).** 1.5B has enough capacity to *prove the loop trains* — to move measurably off the 12.5% hardness gate and approach the RL ceiling on the synthetic split. Confirmed: the small model learns the ask-discover-recommend-permission ritual and its safety gate holds at 0 violations.
- **H0b (competence ceiling).** 1.5B has *too little* capacity to carry the properties the production task actually demands — robust bilingual / code-switched constraint extraction, multi-constraint enumeration under distractors (the 99/100 trap), rehearsal-preserved general-chat retention (H3), and zero-shot transfer to an unseen backend. These are capacity-bound generalization properties, not loop-correctness properties, so validating them requires the larger model.

We therefore *deliberately continued to 7B* as the production line rather than declaring victory on the 1.5B validation. The scale-up is what the shared-reference LoRA trick (§5) exists for: it lets a 7B policy *and* its frozen reference fit in the same 32 GB the 1.5B validation used, so the hardware envelope does not change — only the capacity does. The methodology is the point: *validate the machinery small, then spend capacity where generalization actually lives*.

Hardness gate (pre-registered). Before any training spend, the untrained base scores **12.5%** on the held-out hard split ($n=64$) — inside a pre-registered 10–40% acceptance band (enough signal to bootstrap, enough headroom for RL to matter). A 90%-zero-shot task would not have justified this pipeline.

Algorithms. RLOO (bring-up) vs GRPO (comparison) at an equal 400-trajectory budget.

7 Results

Stage	Task success (es-en hard split)
Base model (hardness gate)	12.5%
SFT (env-recorded multilingual demos)	48.4% (RL-difficulty); 98–100% (standard)
RLOO, 50 steps	92.2%
GRPO, equal 400-trajectory budget	100.0%
GRPO + 4-epoch clipped reuse	100.0% (KL max 0.027)

Table 1: Learning progression. Every arm is violation-free; every number is held-out.

- **H1 — GRPO instability is regime-dependent, not intrinsic.** Measured KL 0.013 (healthy-variance regime) vs 0.58–0.99 drift on a *saturated* task using the identical stack. Hypothesis and threshold registered before the run.

- **H2 — multi-epoch reuse is sufficient but not necessary.** The reuse arm hit the same 100% ceiling as single-pass advantage scaling; reuse added no task success beyond scaling, and is effectively free in wall-clock since rollout dominates (§8). The mechanism-level finding, not the top-line number, was the target.
- **H3 — forgetting is rehearsal-mitigable at no cost (7B).** Task-only SFT loses general-chat retention (12.5%→10.0%); mixing 25% self-distilled general-chat into the same SFT holds retention at 97.5% *and* exceeds the specialist’s shopping competence (0.859 vs 0.820) after identical RL. Rehearsal dominated on both axes.
- **Zero-shot transfer (WebShop).** Unseen products, different backend, no fine-tuning: **0 violations** (safety transfers completely), **60%** task success (competence transfers partially) — an honest domain-shift datapoint.
- **Preemption safety.** A live SIGTERM → Slurm requeue → resume drill produced a clean 20/20-step continuation with KL continuous across the kill seam — measured operationally, not just asserted from checkpoint code.

8 Systems and Efficiency (SS0–SS13)

A 14-row profiling campaign measured every proposed optimization before adopting or rejecting it.

- **Rollout is 95.8%** of RL-iteration wall-clock (3 consistent passes) — effort belongs in rollout, not the optimizer step.
- **93.2s → 13.4s per iteration** ($6.96\times$) via batched lockstep rollouts ($4.98\times$ tokens/iter, $6.4\times$ episodes/min), merge-at-sync vLLM serving ($2.3\times$ alone; also the Volta LoRA-serving workaround), and checkpoint-off updates at small batch ($1.48\times$).
- **Measured and rejected:** engine swap alone ($1.06\times$), prefix caching (Volta-gated and $\leq 7\%$ /turn at these lengths anyway), request packing (8.2% waste), and a custom Triton kernel (profiled ceiling 0.84% of iteration → *deliberately not written*). The profile is the justification for *not* building — as load-bearing as the optimizations adopted.

9 Vision: Agents over a Self-Hosted Distributed Retrieval Substrate

PennyPilot’s **search** is served today by a synthetic catalog — a controlled index that keeps reward verifiable. The next step is to replace that index with a *real* retrieval substrate without losing what made the loop honest.

We build that substrate separately as **Aether**: a from-scratch, single-operator **distributed spatial-vector search engine in Rust** — sharded **shard-nodes** behind a **coordinator**, gRPC transport, leader election with **automatic failover**, cross-cluster **federation**, a natural-language query layer, and **provenance-carrying briefings**. Aether is operational end to end (a running cluster demonstrates coordinator-driven failover and query fan-out across shards).

The vision is their composition: **PennyPilot as the policy, Aether as the environment/retrieval tier**. The agent’s structured **search** fans out over Aether’s sharded index instead of a synthetic catalog; constraint filters become Aether’s structured filters (which aggregate without shipping raw values); the final briefing inherits Aether’s provenance trail, so a recommendation

traces to the exact shards and records that produced it. This turns a controlled RL benchmark into a **self-hosted agentic-retrieval system** — an LLM policy that reasons and asks, grounded on a fault-tolerant distributed index a single practitioner can own. Making the reward verifiable against a *live, distributed* ground truth (rather than a static synthetic one) is the central open problem this composition raises — and the direction we intend to pursue.

9.1 Integration roadmap

We plan the composition in gated phases, each preserving the property that made the loop honest — a *verifiable* reward — while relaxing one assumption at a time:

1. **Backend swap behind the same action.** Route the policy’s structured `search` through an adapter to Aether’s gRPC query API instead of the synthetic catalog. The policy interface (§2.1) is unchanged; only the environment adapter differs, so no retraining is required to switch substrates.
2. **Structured filters and grounded reward.** Map the hidden-constraint predicates (budget ceiling, brand, min-RAM, ...) onto Aether’s structured filters and score `value.quality` against the records Aether returns, so reward is computed on real retrieved specifications rather than a static answer set.
3. **Streaming ingestion (live catalogs).** Feed live product streams into Aether’s sharded index so the catalog is *live*, not frozen — the streaming component the current single-machine, deterministic data pipeline deliberately omits. This is the point at which an ingestion/streaming layer becomes justified, and it lives in the retrieval substrate (Aether), never in the RL loop.
4. **Snapshot-versioned verifiable reward.** A live index changes between the rollout that generated a trajectory and the moment it is scored — a retrieval-side analogue of policy staleness. We will version index snapshots so a trajectory is always scored against the exact catalog state it acted on, keeping the reward reproducible and un-hackable even over a moving ground truth. This is the core research contribution of the composition.
5. **Provenance-carrying recommendations.** The agent’s final briefing inherits Aether’s provenance trail, so every recommendation is traceable to the exact shards and records that produced it — auditable end to end.

Phases 1–2 are engineering behind existing interfaces; phases 3–5 are the open research and the reason the two systems are worth composing rather than merging.

10 Limitations (honest)

- Scripted user simulator has cosmetic phrasing quirks; a frozen-LLM user model is a planned drop-in behind the same seam.
- Mid-flow ground-truth corrections (live budget revision) are deferred pending scenario-schema support — not implemented, not faked.
- WebShop eval uses a dialect-faithful in-repo backend verified against the real parser; the princeton-nlp instance remains a documented, not-yet-wired backend.

- vLLM LoRA serving / prefix caching are Volta-gated; merge-at-sync is the documented workaround with correctness asserts.
- H2’s registered mechanism (multi-epoch reuse) was not why GRPO reached its ceiling — advantage scaling alone sufficed; the reuse arm’s own benefit is an open follow-up.
- Extension reward terms (need-coverage, state-consistency, trajectory-efficiency) were not implemented; the shipped reward is the verifiable base term plus information gain, whose visible cost is a measured tendency to over-ask.
- No preference-optimization stage (DPO/RLHF-style) despite collected thumbs-up/down feedback — the single largest gap to a “complete” modern pipeline.

11 Conclusion

PennyPilot shows that rigorous RL post-training — pre-registered hypotheses, measured hardness gating, honest negative/partial results, and a *verified* (not assumed) safety mechanism — is achievable end to end on a single academic GPU on Volta-class hardware. The central methodological claim is not any individual number in §7, but the discipline connecting them: every reported result was predicted before it was measured, and every reported optimization was adopted or rejected on a profile, not intuition.